

count with array

Array	{ }	{1,4,3,1,2,0,4,2}
Name of array	frequency	member
Length	5	8
Value	frequency[i]	member[i]<frequency.size()
for loop	frequency output	meaning
for(size_t i=0;i<member.size();i++){ frequency[i]++;}	{1,1,1,1,1}	Every element ‘zero’ add ‘1’.
for(size_t i=0;i<member.size();i++){ frequency[mamber[i]]++;}	{1,2,2,1,2}	To count appearance of every element of array ‘member’ at array ’frequency’.
for(int &i:member){frequency[i]++}	{1,2,2,1,2}	Every element value of array ‘member’ refer to the index of the array ‘frequency’ and the index add ‘1’.

```
1 #include <array>
2 #include <iostream>
3 using namespace std;
4
5 int main(){
6     const size_t arrayASize=5;
7     const size_t arrayBSize=8;//longer than arrayASize
8     const size_t arrayCSize=3;//shorter than arrayASize
9     const size_t arrayDSize=5;//value is bigger than arrayASize
10
11     array<int,arrayASize>arrayA1_1={};
12     array<int,arrayASize>arrayA2_1={};
13     //array<int,arrayASize>arrayA3_1={};
14     array<int,arrayASize>arrayA1_2={};
15     array<int,arrayASize>arrayA2_2={};
16     //array<int,arrayASize>arrayA3_2={};
17     array<int,arrayBSize>arrayB1={1,4,3,1,2,0,4,2};
18     array<int,arrayBSize>arrayB2={1,4,3,1,2,0,4,2};
19     array<int,arrayCSize>arrayC1={1,4,3};
20     array<int,arrayCSize>arrayC2={1,4,3};
21     //array<int,arrayDSize>arrayD1={1,8,3,11,8};
22     //array<int,arrayDSize>arrayD2={1,8,3,11,8};
23
24     for(size_t i=0;i<arrayB1.size();i++){arrayA1_1[arrayB1[i]]++;}
25     for(size_t i=0;i<arrayC1.size();i++){arrayA2_1[arrayC1[i]]++;}
26     //for(size_t i=0;i<arrayD1.size();i++){arrayA3_1[arrayD1[i]]++;}
27
28     cout<<"arrayA1_1: ";
29     for(int &i:arrayA1_1) cout<<i<<" ";
30     cout<<endl;cout<<"arrayA2_1: ";
31     for(int &i:arrayA2_1) cout<<i<<" ";
32     cout<<endl;
33     //for(int &i:arrayA3) cout<<i<<" ";
34 }
```

Built–
int

example .cpp

in

```
34
35
36     for(int &i:arrayB2)arrayA1_2[i]++;
37     for(int &i:arrayC2)arrayA2_2[i]++;
38     //for(int &i:arrayD2)arrayA3_2[i]++;
39
40     cout<<"arrayA1_2: ";
41     for(int &i:arrayA1_2) cout<<i<<" ";
42     cout<<endl;cout<<"arrayA2_2: ";
43     for(int &i:arrayA2_2) cout<<i<<" ";
44     cout<<endl;
45     //for(int &i:arrayA3) cout<<i<<" ";
46
47     return 0;
48 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(base) penggenpei@penggenpeideMacBook-Pro ~ % cd Desktop/c++/0424
乗後
(base) penggenpei@penggenpeideMacBook-Pro 0424課後 % g++ array0-2
.cpp -o program
(base) penggenpei@penggenpeideMacBook-Pro 0424課後 % ./program
arrayA1_1: 1 2 2 1 2
arrayA2_1: 0 1 0 1 1
arrayA1_2: 1 2 2 1 2
arrayA2_2: 0 1 0 1 1
(base) penggenpei@penggenpeideMacBook-Pro 0424課後 %

example .cpp

array

main(int argc,char* argv[]){}

```
char* argv[ ]
={ &argv[0], &string1[0], &string1[0],...}
={ argv, &char1-1, &char2-1,...}
```

Syntax	Difinition	Sort	Meaning
char*	A "pointer" points to the first character of a string.	type	Begining "address" of a string.
argv[]	Including elements, each element is a pointer.	array	An array "made of address" of strings' first character.

```
argv=&argv[0]:
```

The address of the first element of the subordinate array in another hand, &array[0] is assigned to The name of an array.

value	content type stored		content inputed in terminal	content type stored in memory
argv[0]	char1*	array's begining memory addreass	Name of "main" function, not wit .cpp	char
argv[1]	char2*	The first string's begining memory address.	The first command	char
argv[2]	char*3	The second string 's begining memory address.	The second command	char

If command is number:

```
Standard Library: sctdlib
tool:             atof,atoi
deal:             content inputed in terminal
mathod:           translate with ASCII
```

const usage	address pointed to	content pointed to in memory	example
int *countPtr;	can be change	can be change	void displayArray(int *array, size_t size);
const int *countPtr;	can be change	can't ba change	void displayArray(const int *array, size_t size);

const usage	address pointed to	content pointed to in memory	example
<code>int *const countPtr = &x;</code>	can't be change	can be change	<code>void displayArray(int *const array, size_t size);</code>
<code>const int *const countPtr = &x;</code>	can't be change	can't be change	<code>void printArray(const int *const array, size_t size);</code>

int	content type	meaning	*	ex
<code>int *n</code>	address	claim a pointer "n" to store an address of integral	turn the type of "n" to be a pointer	0x61ff08
<code>a</code>	integral	integral		7
<code>&a</code>	address	memory address of integral "a"		0x61ff08
<code>n</code>	address	a pointer stores memory address of "a"		0x61ff08
<code>*n</code>	integral	"a" : point to content of "n"	point to the address stored by "n" to get content inside it	7

```
void fun1(int *n)
fun1(&a)
```

	content type	meaning	*	ex
<code>int *n</code>	address	claim a pointer "n" to store an address of integral	turn the type of "n" to be a pointer	0x61ff08
<code>a</code>	array→address	name of an array and turn to be <code>&a[0]</code>		{ x1 , x2 , x3 , ...}→x1
<code>&a[i]</code>	address	memory address of integral <code>a[i]</code>		0x61ff08
<code>n</code>	address	a pointer stores memory address of <code>&a[0]</code> or <code>a[]</code>		0x61ff08
<code>*n</code>	integral (<code>a[0]</code>)	"a" : point to content of "n"	point to the address stored by "n" to get the content inside it	x1

put `&a[0]`: `&a[...]`:memory address of the whole array,not a integral.

```
void fun1(int *n);
fun1(a);
```

```
void fun1(int *n);
fun1(&a[0]);
```

array	content type	meaning	*	ex
<code>int *n[]</code>	address	claim pointers " <code>n[i]</code> " to store addresses of integrals	turn the type of " <code>n[i]</code> " to be a pointer	0x61ff08
<code>a</code>	integral	integral to be an element		7
<code>&a</code>	address	memory address of integral " <code>a</code> "		0x61ff08
<code>n[i]</code>	address	a pointer stores memory address of <code>&n[i]</code>		0x61ff08
<code>*n[i]</code>	integral	" <code>a</code> " : point to content of " <code>n[i]</code> "	point to the address stored by " <code>n[i]</code> " to get the content inside	7

	content type	meaning	*	ex
<code>char *argv[]</code>	array for address	claim pointers " <code>argv[i]</code> " to store addresses of chars	turn the type of " <code>argv[i]</code> " to be a pointer	0x61ff08
<code>a</code>	the string	name of function " <code>main</code> " or another command from terminal		<code>./program</code>
<code>&a</code>	address	memory address of any command from terminal		0x61ff08
<code>argv[i]</code>	address	a pointer stores memory address of any command		0x61ff08
<code>*argv[i]</code>	the first char of string	" <code>a</code> " : point to content of " <code>argv[i]</code> "	point to the address stored by " <code>argv[i]</code> " to get content inside it	.

```
code editor:
int main(int argc, char *argv[]){
    return 0;
}
```

```
terminal:
./program 2.0 0.5
```

```
command: ./program, 2.0, 0.5
type: string
```

Offset notation

A parameter "a" 's type is unknown.
Its address: &a
Claim its pointer: void *n

"void *n" equals to "&a" equals to "n"

If there are two claim:
int *p=&b;
void *n;
n=p;
p!=n;
int *p=reinterpret_cast<int*>(n);

The knowable can be assigned to the unknown.
The unknown can;t be assigned to the knowable.

An array "a[n]"

address	calulation	equals to	meaning	unit	content(value)	meaning
a	a+number	&[0+number]	&a[0]+number*4	1 element / 4 bytes	*(a+number)	a[number]

example:

ptr=&a[0]

(ptr+3)=a[3]=ptr[3] :(&a[0]+3*4)=*(&a[3])=*(address+3*4)=*(a+3)

*ptr+3=&a[0]+3 :a[0]+3

value	write method	operation
a[3]	a+3	legal
a[0]=a[3]	a+=3	illegal